Aiori Hackathon 3
Advanced Internet Operations Research in India

PS-022

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Kernel-level protection against high-volume Dns query floods using eBPF/XDP with per sourcer and global rate limiting



Team Code : A3-PS022-TC063

Team Name : Ping@St.Josephs

Team Members :

1. Balachandhar D
2. Buvanesh D
3. Dineshkumar A (Mentor)

Problem Description

Key Issues in DNS Query Flood Handling

- High-Volume DNS Query Floods: Excessive UDP DNS packets can overwhelm Linux servers.
- High CPU Utilization: Processing large numbers of packets consumes significant CPU resources.
- Kernel Resource Overhead: Packet processing and socket-buffer handling increase system overhead.
- DNS Response Delays: Legitimate queries may experience increased latency and timeouts.
- Reduced Service Availability: Excessive traffic can degrade DNS service performance for legitimate users.
- Lack of Early Filtering: Unwanted traffic needs to be limited before it passes through much of the normal Linux networking stack.

Background

Technical Background of DNS Query Flood Handling

- **DNS over UDP:** DNS commonly uses UDP to exchange queries and responses between clients and DNS servers.
- **Linux Network Processing:** Incoming packets are processed by the Linux networking stack before reaching the DNS application.
- **Kernel Resource Overhead:** High packet rates can increase CPU usage and packet-buffer processing overhead.
- **eBPF Technology:** eBPF enables verified programs to execute within the Linux kernel for programmable packet processing.
- **XDP Packet Filtering:** XDP provides an early packet-processing hook that can pass or drop packets before much of the normal networking stack processes them.
- **Rate-Limiting Approach:** Per-source and global limits can control excessive DNS traffic while allowing eligible requests to continue.

Literature & RFC Review

1. RFC 1035 — Domain Names

P. Mockapetris · 1987

- Defines DNS message formats, queries, responses, and protocol operation.
- It provides the foundation for understanding DNS packets.

Reference: RFC 1035

2. eBPF Documentation

- Explains eBPF programs, verification, and their execution within the Linux kernel.
- This supports the design of our programmable packet-filtering mechanism.

Reference: Linux eBPF documentation

3. XDP Early Packet Processing

- XDP supports packet actions such as passing, dropping, and redirecting incoming packets.
- With native driver support, it can act before socket-buffer allocation, making it relevant to early DNS flood filtering.

Reference: XDP program type documentation

4. nftables Baseline

- nftables supports packet and byte rate limits.
- We will use it as a conventional Linux filtering baseline and compare its results with our eBPF/XDP implementation.

Reference: nftables rate-limit documentation

Proposed Solution



Early Packet Filtering
Use eBPF/XDP to inspect incoming DNS packets at an early stage and drop packets that exceed the configured limits.



Per-Source Rate Limiting
Apply configurable limits to DNS queries from individual source IP addresses to control excessive traffic.



Global Rate Limiting
Enforce an overall DNS traffic limit to help prevent excessive packet rates from overwhelming the server.



DNS Server Integration
Forward permitted packets to the DNS server while dropping excess traffic before it reaches normal packet processing.

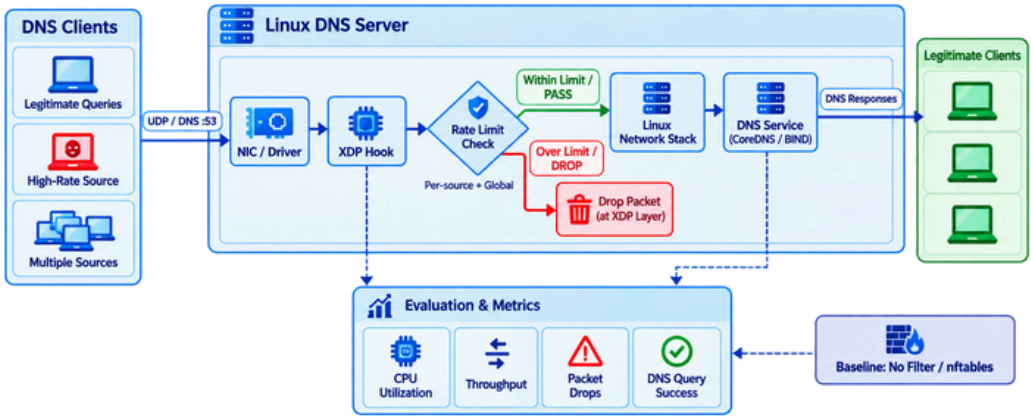


Performance Evaluation
Compare XDP filtering with no filtering and an nftables baseline using CPU utilization, throughput, packet drops, and legitimate DNS query success.

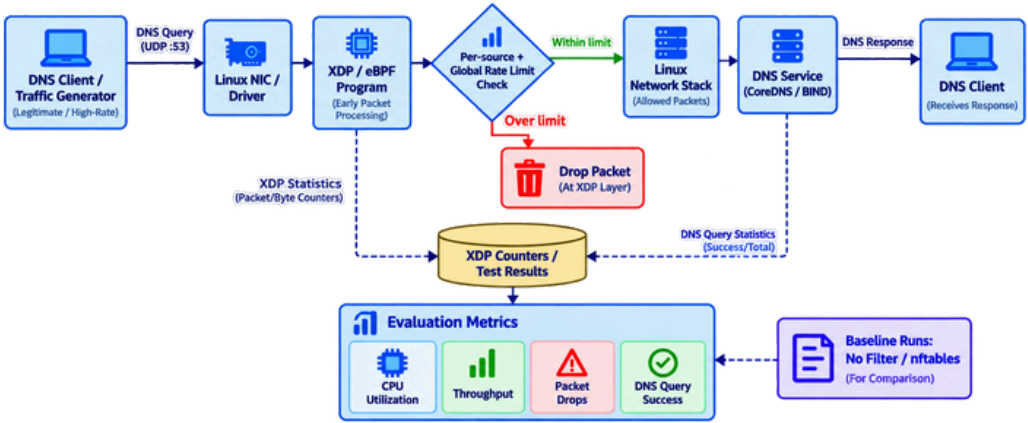
Optimization Proposed

Aspect	Before Optimization	After Proposed Optimization
Packet Filtering	Packets processed through the normal network stack	XDP filters packets at an early stage
Rate Limiting	No project-specific XDP limits	Per-source and global rate limits
Excessive DNS Traffic	May increase CPU and kernel overhead	Excess packets can be dropped early
DNS Server Load	Unwanted traffic may reach the DNS server	Only permitted packets proceed to the DNS server
Performance Monitoring	Baseline measurements required	Compare CPU usage, throughput, packet drops and DNS success

Solution Architecture & Design



Data Flow Diagram



Deployment Timeline

#	Project Milestone	Date	Description / Deliverable
1	Requirement Analysis & Environment Setup	16 Oct – 17 Oct	Define objectives, set up Linux lab, configure DNS server and testing tools
2	Design & Implementation (eBPF/XDP)	18 Oct – 21 Oct	Develop XDP program, configure per-source and global rate limits
3	DNS Traffic Generation & Testing	22 Oct – 24 Oct	Test with normal and high-rate DNS queries, multi-source traffic
4	Baseline Comparison (No Filter / nftables)	25 Oct – 27 Oct	Configure nftables, conduct comparison tests and collect performance data
5	Performance Evaluation	28 Oct – 29 Oct	Analyze CPU usage, throughput, packet drops and DNS query success rate
6	Final Integration, Documentation & Demo	30 Oct – 31 Oct	Integrate the complete system, prepare report and final demonstration

Bibliography I

[1] [P. Mockapetris, **"Domain Names—Implementation and Specification,"** RFC 1035, Internet Engineering Task Force, Nov. 1987. Available: *https://www.rfc-editor.org/rfc/rfc1035.html*

[2] Linux Kernel Documentation, **"BPF Documentation."** Available: https://docs.kernel.org/bpf/

[3] eBPF Docs, **"Program Type BPF_PROG_TYPE_XDP."** Available: *https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_XDP/*

[4] **nftables Project,** "Limits." Available: https://wiki.nftables.org/wiki-nftables/index.php/Limits

[5] nftables Project, **"Rate Limiting Matchings."** Available: https://wiki.nftables.org/wiki-nftables/index.php/Rate_limiting_matchings

[6] eBPF Docs, **"eBPF on Linux."** Available: https://docs.ebpf.io/linux/

THANK YOU

Thank you for being part of our journey toward smarter and more resilient DNS protection